

SENG 383 Project

Mustafa Eren Toktaş - Student A

Hayri Melih Alan - Student B

<https://github.com/hmelihalan/SENG383-3-project>

<https://youtu.be/L5SFOerzjog>

2. Final Design Artifacts:

● **Class Diagram:**

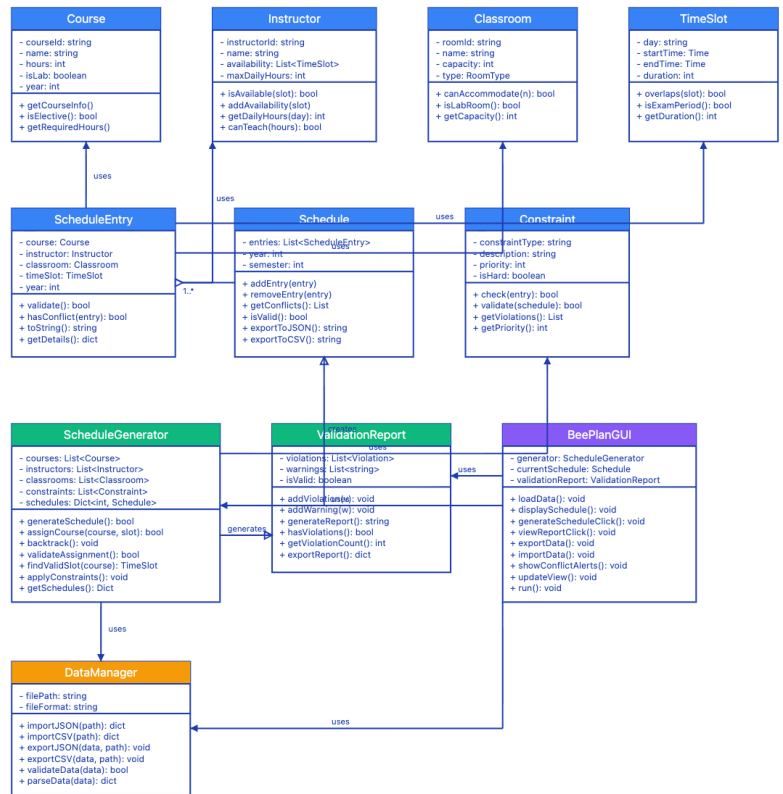
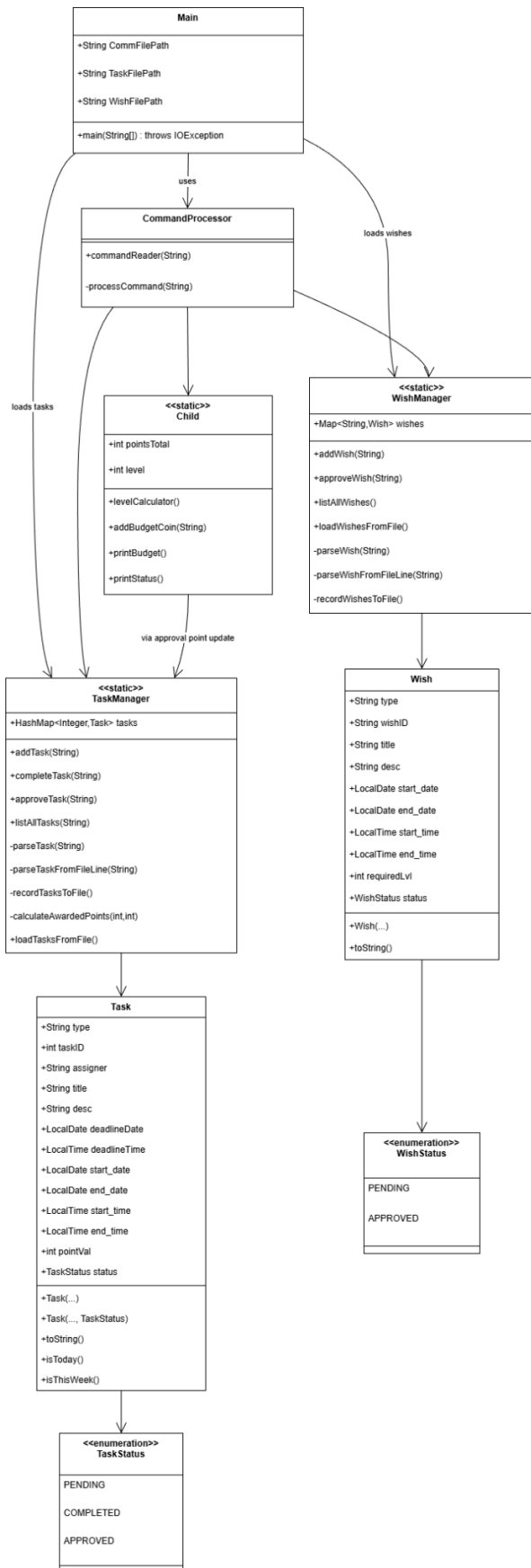


Image 1: Bee Plan Class Diagram Figma

Image 2: Kid Task Class Diagram Canva

- **Activity Diagram:**

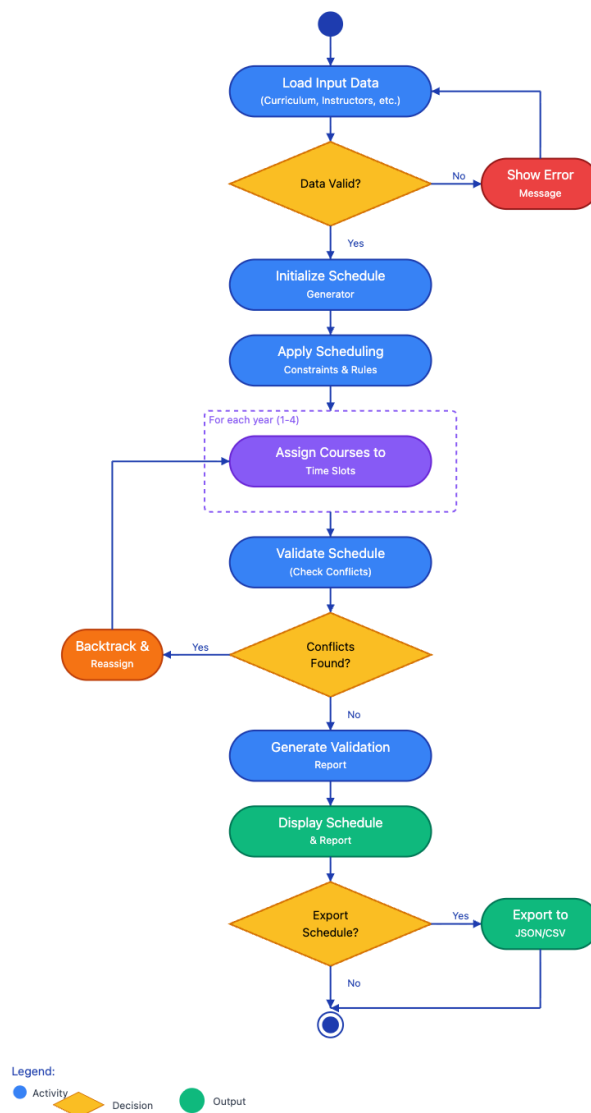


Image 3: BeePlan Activity Diagram Figma

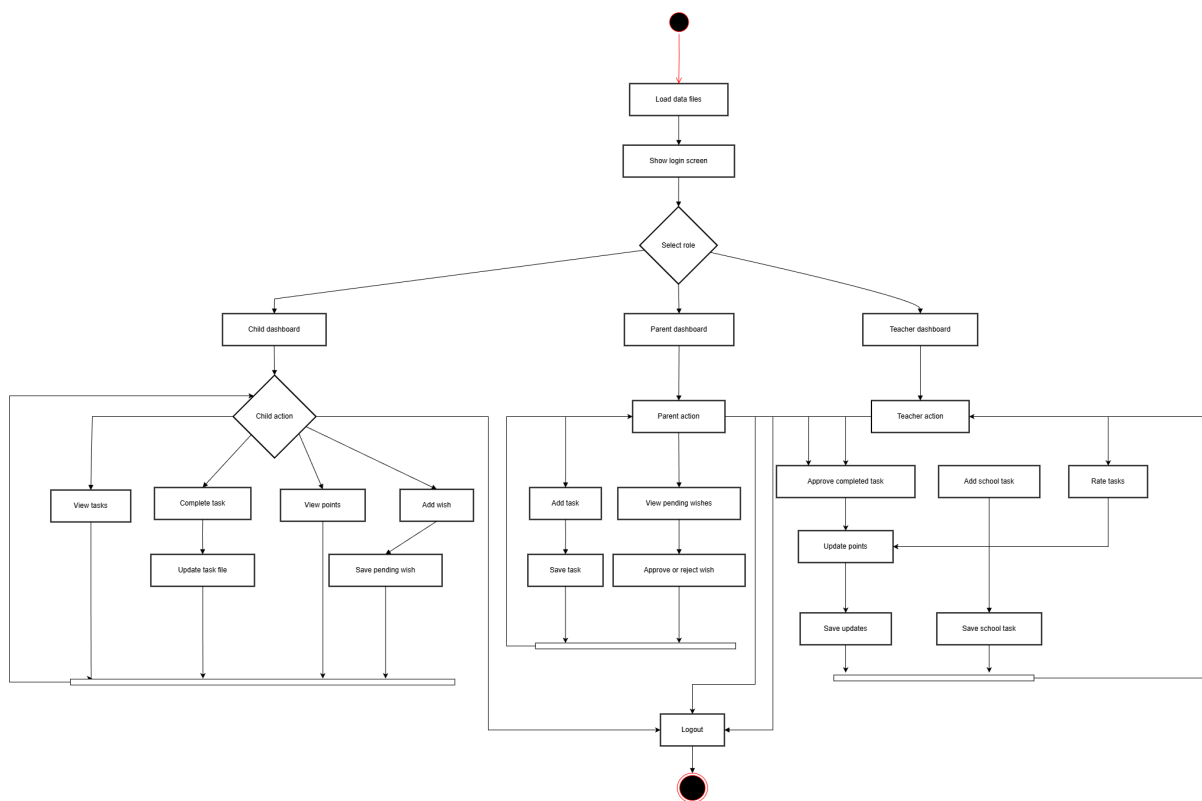


Image 4: KidTask Activity Diagram Canva

- GUI Screenshots:

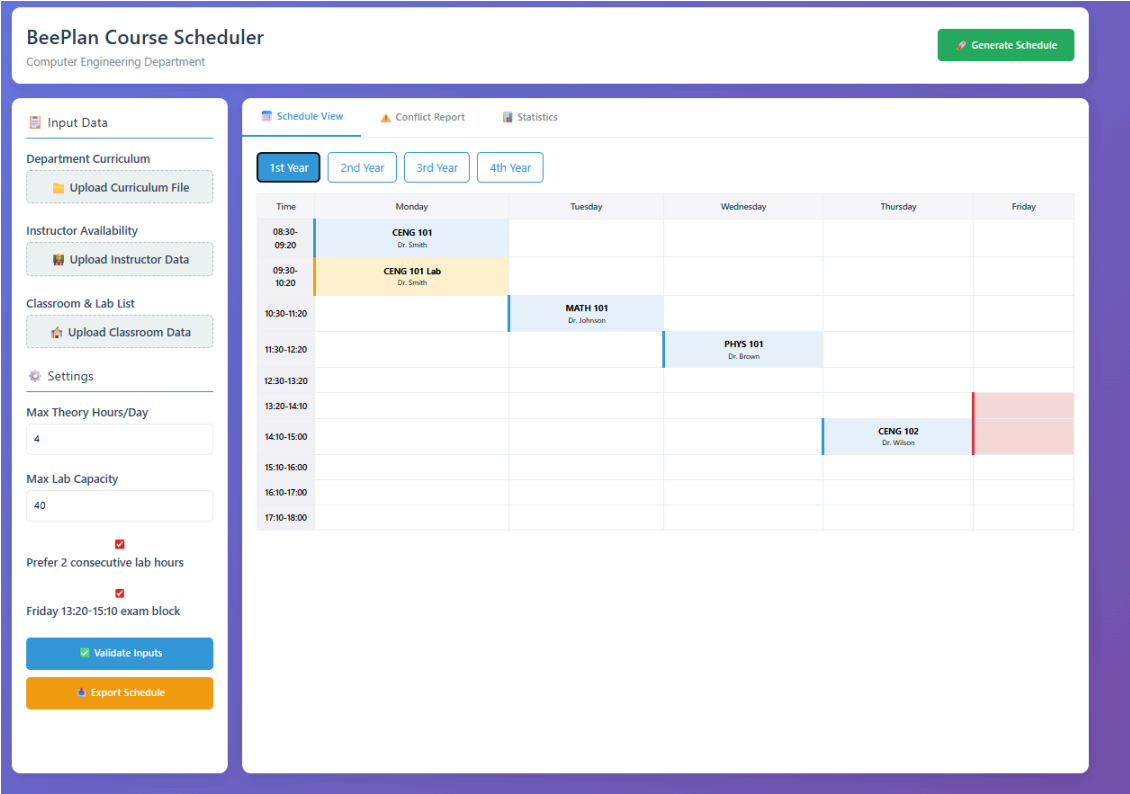


Image 5: BeePlan Main Page Design

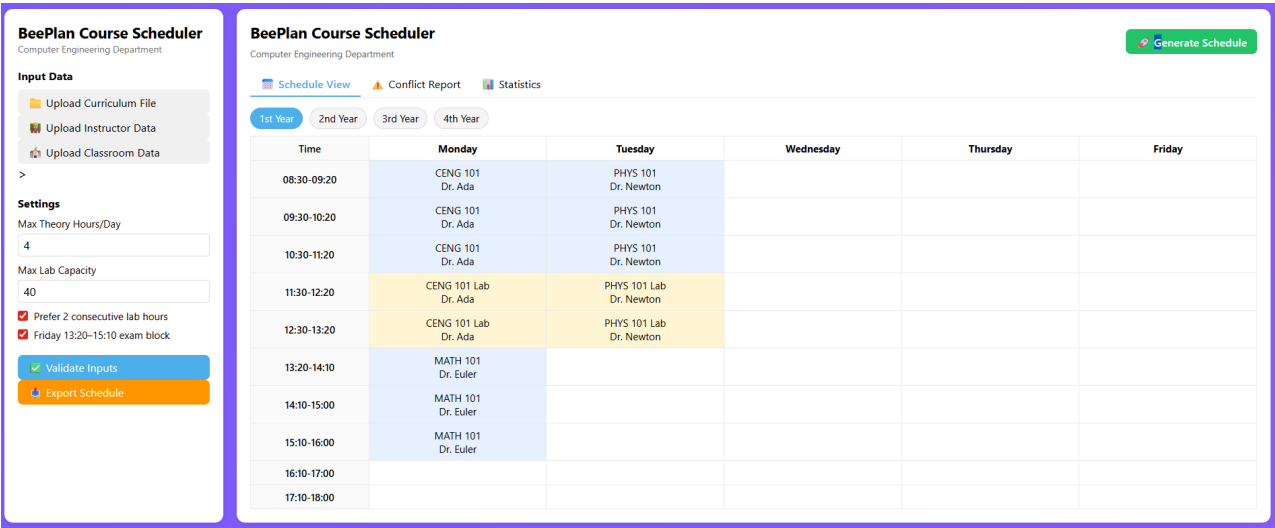


Image 6: BeePlan Main Page Output Program

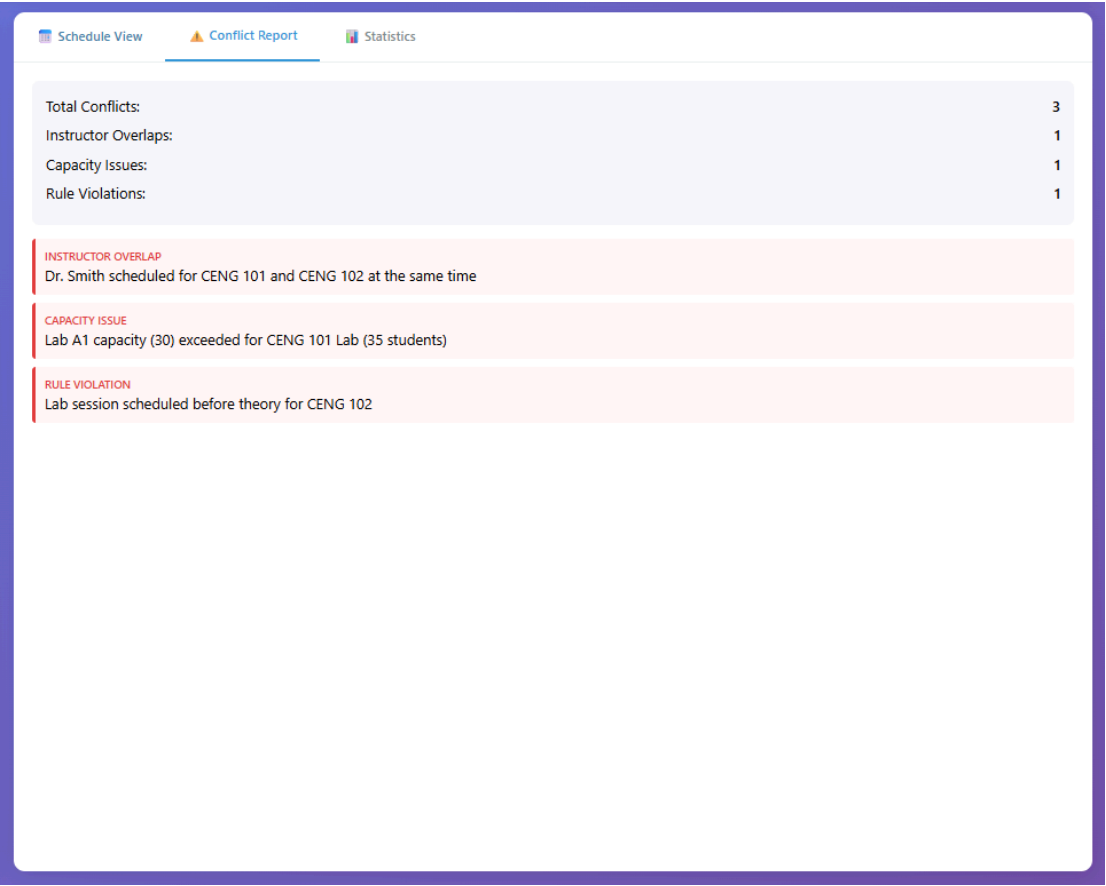


Image 7: BeePlan Conflict Report Design

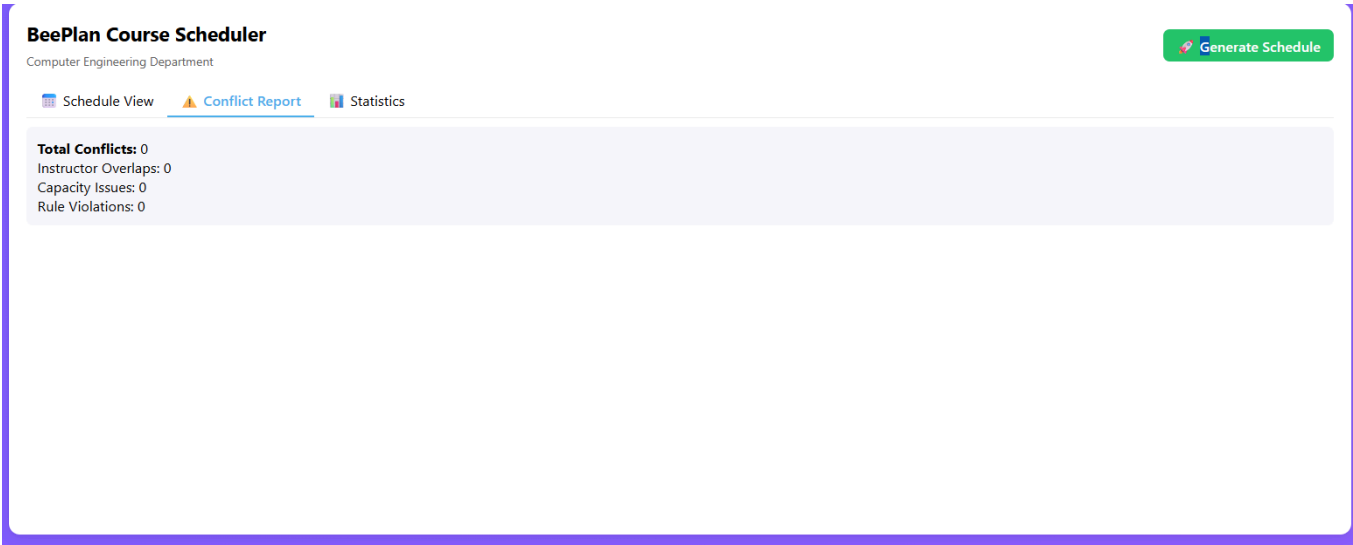


Image 8: BeePlan Conflict Report Output Program

Schedule View Conflict Report Statistics		
Total Courses Scheduled:		
		24
Theory Hours:		
		18
Lab Hours:		
		6
Classroom Utilization:		
		78%
Schedule Efficiency:		
		92%

Image 9: BeePlan Statistics Design

BeePlan Course Scheduler		
Computer Engineering Department		
Schedule View Conflict Report Statistics		
Total Courses Scheduled: 18		
Theory Hours: 33		
Lab Hours: 14		
Classroom Utilization: 50%		
Schedule Efficiency: 85%		

Image 10: BeePlan Statistics Output Program

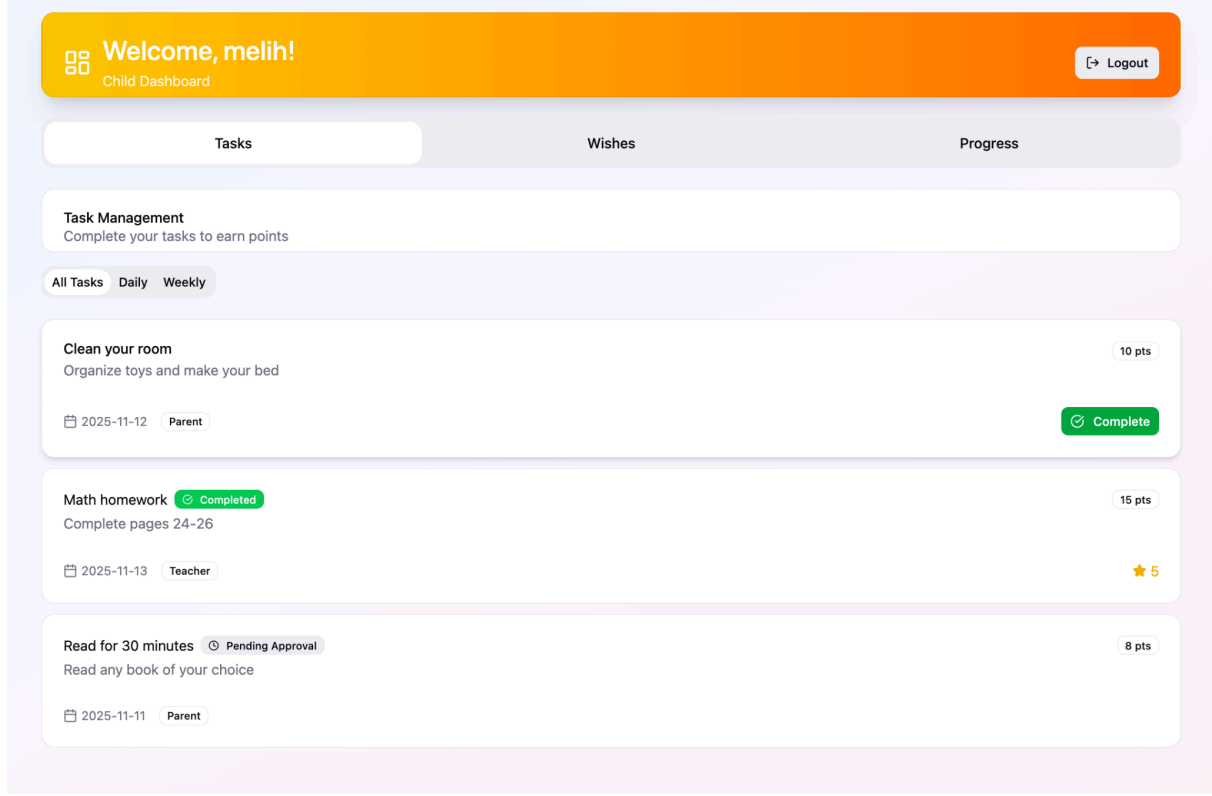


Image 11: KidTask Tasks Design



Image 12: KidTask Tasks Output Program

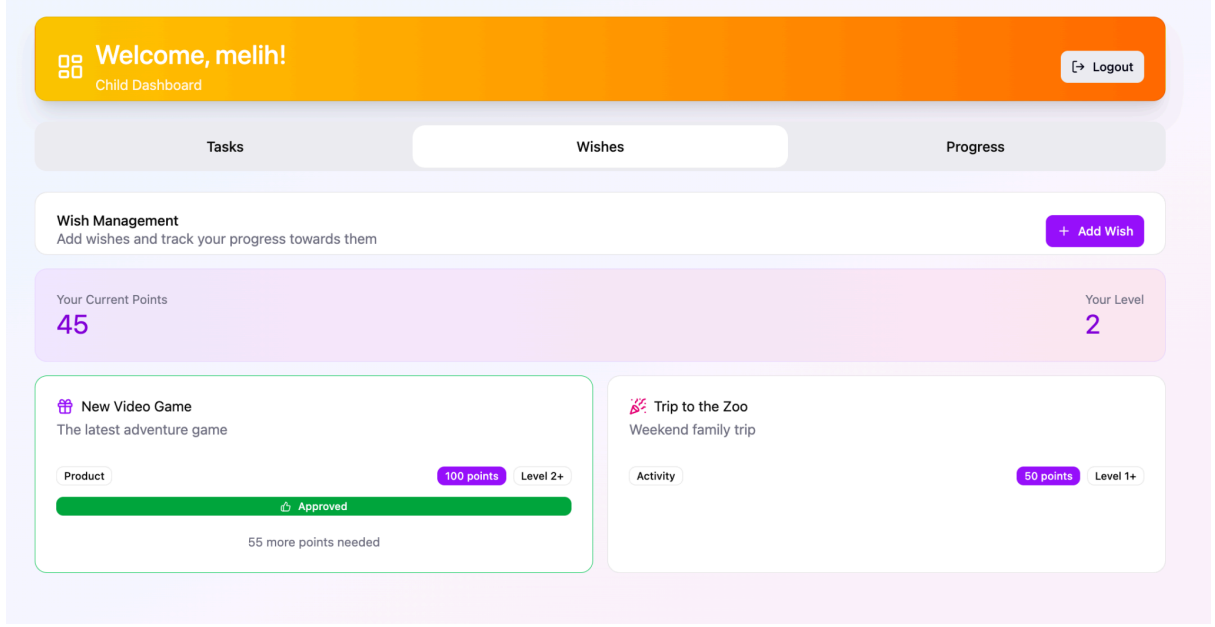


Image 13: KidTask Wishes Design



Image 14: KidTask Wishes Output Program

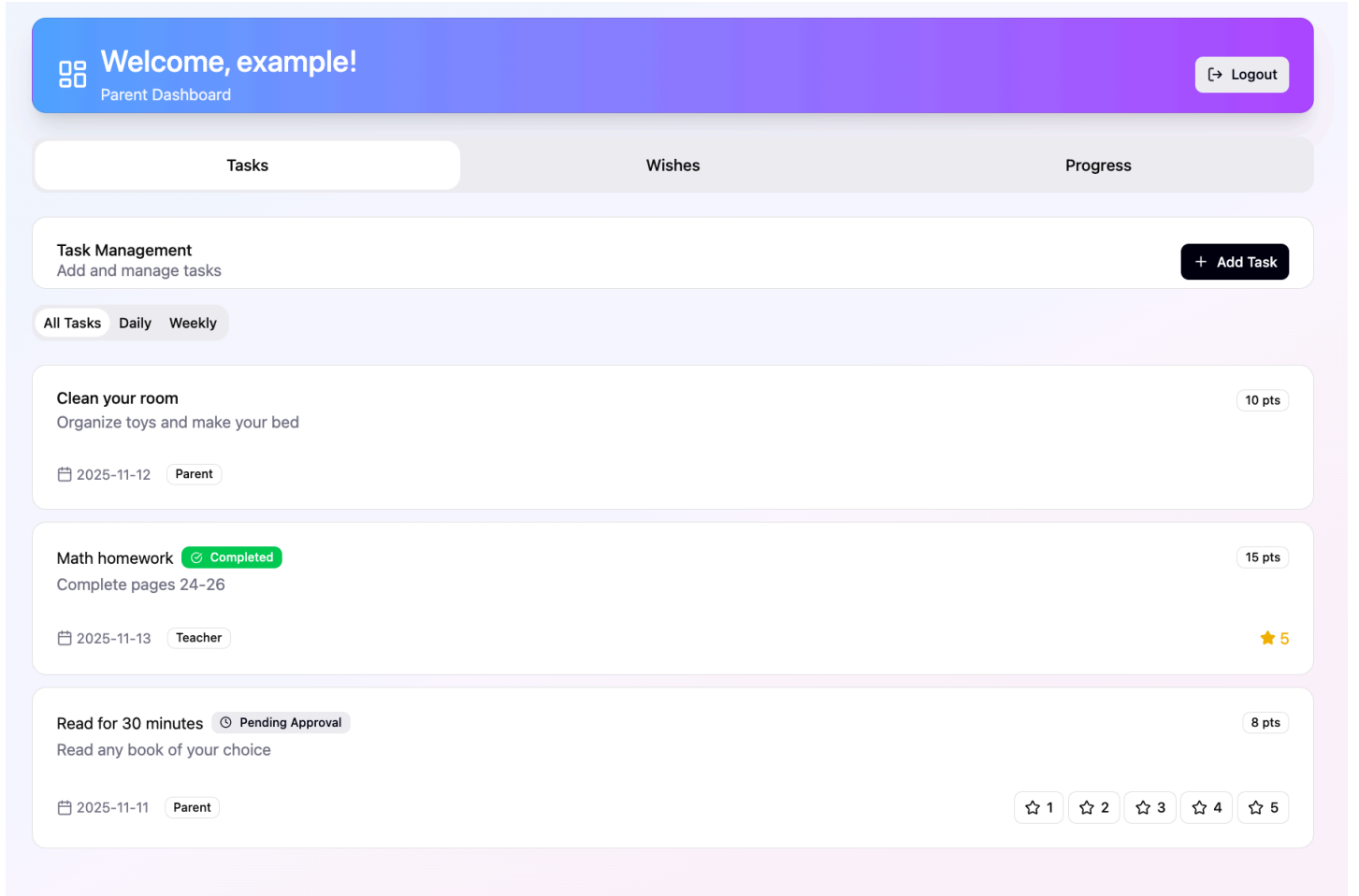


Image 15: KidTask Task Management Design



Image 16: KidTask Task ManagementOutput Program

3. AI Usage & Prompts

Process	Tool Used	Prompt	AI Output Analysis	Human Intervention (Revision)
GUI Design	Figma (LLM-assisted)	“Create a complete GUI design for the KidTask application, including Child, Parent, and Teacher roles.”	All required screens were generated and the overall visual consistency was high. However, the activity diagram was weak, and the class diagram became overly complex.	The activity diagram was simplified, and the class diagram was manually reorganized by removing unnecessary relationships.
Coding	GitHub Copilot (Coding Task)	“I will send you UI mockup images and then ask you to write GUI code for this program based on those images (probably Swing-based).”	Main, LoginPanel, and DashboardPanel classes were generated. Role-based tabs were implemented according to the UI design. However, the ID generation logic caused data overwriting issues.	The generated code was integrated into the project and tested. The ID-generation issue was identified and reported for correction.

Debugging & Logic Fix	Gemini (Debugging Task)	“There is an error. I seem to only be able to add 2 wishes at most. When I try to add a third wish, it deletes the previous wish.”	The issue was correctly identified as an ID collision problem. A safer algorithm using <code>maxId + 1</code> instead of <code>size() + 1</code> was proposed and implemented.	The corrected logic was manually merged into the relevant methods, resolving the data overwrite issue.
GUI Modification	GitHub Copilot (Coding Task)	“Remove the ‘Progress’ section from each of the user GUIs.”	The Progress tab was removed from all user interfaces. The updated DashboardPanel class also included previous bug fixes.	The updated class was reviewed and integrated into the project without further changes.
UI Implementation	Figma (Design-to-Code Task)	“Can you implement this application based on the provided UI images and functional requirements?”	The UI was implemented correctly according to the designs. Minor issues were observed in fonts and text alignment.	Menu labels and font-related issues were manually corrected to improve visual consistency.
GUI Design	Canva (Design Task)	“Can you create a UI design wireframe from the provided SRS and source code files?”	The generated wireframes were complete and logically consistent. No major design issues were detected.	No revision was required. Minor adjustments were made while exporting the designs from Canva.

Backend & Scheduling Verification	DeepSeek (Testing / V&V Task)	“Upload mutually compatible curriculum, instructor availability, and classroom JSON files and verify that the generated schedule complies with all defined scheduling rules.”	The system successfully generated schedules from multiple JSON inputs. However, instructor availability and classroom constraints were not fully enforced in early tests.	JSON schemas were standardized, scheduling rules were refined, and constraint checks were added to ensure full compliance.
-----------------------------------	-------------------------------	--	---	--

4. Verification & Validation

Test Case Table:

Test Scenario	Input / Condition	Expected Result	Actual Result	Status
Task Creation – Duplicate ID (KidTask)	Create a task with an existing ID	System prevents overwrite	Existing task overwritten	Failed

Task Creation – Automatic Unique ID (After Revision)	Automatically generate a unique task ID	Task added successfully	Task added successfully	Passed
Wish Creation – Duplicate ID (KidTask)	Add a wish with an existing ID	System prevents overwrite	Existing wish overwritten	Failed
Role-Based Access: Child → Teacher Page (KidTask)	Child attempts to access Teacher page	Access denied / page hidden	Page hidden	Passed
Schedule Generation – Valid JSON Input (BeePlan)	Upload compatible curriculum, instructor, and classroom JSON files	Valid schedule generated	Schedule generated successfully	Passed
Schedule Generation – Resource Conflict (BeePlan)	JSON files with instructor/classroom conflict	Conflict detected and blocked	Conflict not detected initially	Failed

Bug Fixing with AI Tutor:

During the testing phase, **DeepSeek** and **OpenAI (ChatGPT)** were used as AI tutors to analyze and resolve **two critical bugs**.

Bug 1: Task / Wish ID Collision (Critical)

Problem Description:

When adding new tasks or wishes, the system directly trusted manually provided IDs. This caused newly added items to overwrite existing ones if the same ID was used.

AI Usage:

- **Tool:** ChatGPT / GitHub Copilot
- **Prompt (Summary):**
“Analyze why adding new tasks overwrites existing tasks.”

AI Output Analysis:

The AI correctly identified that the issue was caused by a **non-unique ID generation mechanism**, which led to data collisions.

Human Intervention (Revision):

- Implemented automatic and incremental ID generation
- Added checks to ensure ID uniqueness before insertion

Result:

The bug was fully resolved, and task/wish creation became stable and reliable.

Bug 2: Missing Input Validation and Logical Checks

Problem Description:

Some user actions (such as empty inputs or unauthorized role access) were not properly validated, leading to unexpected behavior.

AI Usage:

- **Tool:** DeepSeek & Gemini

- **Prompt (Summary):**
“Analyze the project for missing validations and logical flaws.”

AI Output Analysis:
The AI accurately detected missing input validations and role-based access issues, providing clear and actionable recommendations.

- Human Intervention (Revision):**
- Added input validation checks
 - Strengthened role-based access control

Result:
User-induced errors were significantly reduced, improving system robustness.

Peer Review Findings:

Reviewer	Issue Identified	Description	Resolution
Student A	GUI–Backend Integration	Some GUI buttons were not properly connected to backend methods, resulting in no action on user interaction.	Event handlers were correctly linked to the related backend functions.
Student B	Code Duplication	Similar logic blocks were repeated across multiple classes, reducing maintainability.	The repeated logic was refactored into shared helper methods.
Student A	Exception Handling	Technical exception messages were displayed directly to the user, reducing usability.	User-friendly error messages were implemented and technical details were logged internally.

Student B	Code Readability	Some variable and method names were ambiguous and did not clearly reflect their purpose.	Variables and methods were renamed to improve clarity and readability.
------------------	---------------------	--	--